FLORIX TECHNOLOGIES

Phone: +91 9986639994
Mail: info@Florixtechnologies.com
Web: www.Florixtechnologies.com

Practical Assignment
Mini Expense Tracker

Position	Full Stack Developer Intern
Expected effort	3–4 hours
Tools needed	A code editor (VS Code), a web browser, Python 3
Deliverables	A web page (HTML, CSS, JavaScript), a Python program, a short README

1. Context

Thank you for taking the time to interview with us. This assignment helps us see how you think and how you write code on your own. It is a small project. You do **not** need to know any frameworks, databases or APIs. Only the basics of HTML, CSS, JavaScript and Python are needed.

A simple, working solution that you fully understand is better than a fancy one you cannot explain.

2. What You Will Build

An **Expense Tracker** is a small tool where a person writes down what they spent money on. You will build it in two parts:

- **Part A:** A web page where the user adds, views and deletes expenses.
- **Part B:** A Python program that reads a list of expenses from a file and prints a summary.
- **Part C:** A short README that explains your work.

3. Part A — The Web Page

Create three files: `index.html`, `style.css` and `script.js`. The page must open directly in a browser by double-clicking `index.html`. No server is needed.

A1. Add an expense

- A form with these fields: **Title** (text), **Amount** (number), **Category** (dropdown with Food, Travel, Shopping, Bills, Other) and **Date**.
- An **Add Expense** button. When clicked, the expense appears in a table below the form.
- The table shows: Title, Amount, Category, Date and a **Delete** button.
- After adding, the form should clear itself.

A2. Check the input (validation)

- Title must not be empty.
- Amount must be a number greater than 0.
- Date must be selected.
- If something is wrong, show a clear message on the page (for example, in red text under the form). Do not use only `alert()`.

A3. Total and filter

- Show the **Total Spent** at the top of the page. It must update when an expense is added or deleted.
- Add a dropdown to filter the table by category (with an “All” option). The total should match what is shown in the table.

A4. Keep the data

- When the page is refreshed, the expenses should still be there.
- Hint: search for "JavaScript localStorage". Store your list as text using `JSON.stringify` and read it back with `JSON.parse`.

A5. Appearance

- Keep it clean and readable. Use your own CSS (no ready-made CSS libraries such as Bootstrap).
- It does not need to be fancy. Spacing, readable fonts and clear buttons are enough.

4. Part B — The Python Program

Create a file named `summary.py`. Save the sample data below in a file named `expenses.csv` (a CSV file is just text with values separated by commas). Use only Python's built-in features, such as the `csv` module. Do not install any packages.

```
title,amount,category,date
Lunch,180,Food,2026-10-01
Bus pass,500,Travel,2026-10-01
Notebook,60,Shopping,2026-10-02
Electricity bill,1200,Bills,2026-10-03
Dinner,250,Food,2026-10-03
Cab,320,Travel,2026-10-04
```

Your program should read the file and **print** the following:

- The total amount spent.
- The total spent in each category.
- The single highest expense (its title and amount).
- The number of expenses recorded.

It must also **write** the same summary into a file named `summary.txt`.

Handle these situations without crashing

- `expenses.csv` does not exist: print a friendly message.
- A row has an amount that is not a number (for example, "abc"): skip that row and print which row was skipped.
- The file has no expenses: print "No expenses found."

Tip: test your program by changing the CSV file yourself (add a bad row, delete the file, empty it).

5. Part C — README

Create a file named `README.md`. Answer in your own words. A few lines each is enough.

- How do I run Part A and Part B?
- What was the hardest part, and how did you solve it?
- What did you learn that was new to you?
- If you had two more days, what would you add or improve?

6. Working Guidelines

- You may use the internet, documentation (MDN, Python docs) and tutorials.

Phone: +91 9986639994
Mail: info@Florixtechnologies.com
Web: www.Florixtechnologies.com

- You may use AI tools to understand concepts. However, you must be able to **explain every line** you submit. In the review, we will ask you to explain your code and make a small change to it live.
- Do not copy a full project from the internet. Write the code yourself.
- Use meaningful names for variables and functions, and add short comments where it helps.
- If something is unclear, make a sensible assumption and write it in your README. You may also ask HR for clarification.

7. What to Submit

- Folder with: `index.html`, `style.css`, `script.js`, `summary.py`, `expenses.csv`, `summary.txt`, `README.md`.
- Upload the folder to a GitHub repository and share the link, **or** send it as a .zip file.
- Optional: 2–3 screenshots of your page inside the README.

8. Bonus (Optional)

Only attempt these after the main work is complete.

- Edit an existing expense.
- Show the total for each category on the web page.
- Sort the table by amount or date.

9. Review

After submission, we will have a short review together. Please be ready to explain how your code works, why you wrote it that way, and how you tested it. We are more interested in your understanding than in a perfect result.